Civilian Contributor Enhancement

Design Specification

WIMS-BFP — Philippines Bureau of Fire Protection

Date: 2026-07-06 · Status: Draft

Defense panel feedback · Design grill · Oracle consultations

Civilian Contributor Enhancement — Design Spec

Date: 2026-07-06 **Status:** Draft **Sources:** Defense panel feedback, design grill with wims-route + grill-with-docs skills, Oracle subagent consultations **Glossary:** See CONTEXT.md (terms: Civilian Contributor, Anonymous Contributor, Registered Contributor, Contributor Trust Score, Routing Distance, Photo Metadata Analysis)

---

1. Problem

The current civilian reporting flow sends reports to a national validator who can only return simple feedback (actioned/bogus). The civilian gets nothing meaningful back from the system — no routing info, no station awareness, no contribution recognition. A defense panel identified three gaps:

- **No feedback loop** — civilians don't know what happened after they report. "Why not just call 911?"
- **No trustworthiness signals** — reports lack photo evidence with verifiable metadata.
- **No contributor identity** — regular reporters have no way to build reputation or track their impact.

---

2. Design Overview

Three interconnected workstreams:

A) Operational feedback via routing — Give civilians immediate, concrete information about which station would respond and estimated arrival time.

B) Photo metadata analysis — Allow photo uploads with GPS verification to increase report trustworthiness.

C) Contributor model — Two paths (anonymous + registered) with different guardrails, feedback richness, and a trust score for registered contributors.

All workstreams share the same core API surface — auth detection determines guardrails and data richness.

---

3. Three-Tier Feedback

Tier	Audience	What they see
1 — Tracking page	Anyone with report ID (no auth)	Report status, station name/phone, road distance, ETA range
2 — Registered dashboard	Authenticated CIVILIAN_REPORTER	Full report history, trust score, contribution stats, per-report routing + photos
3 — Validator panel	NATIONAL_VALIDATOR	Routing data + photo metadata flags + ability to enrich feedback

The tracking page (GET /api/civilian/reports/{id}) already exists. Tiers 2 and 3 are new.

4. Routing Data (OSRM Integration)

4.1 Data Source

OSRM Public API (router.project-osrm.org) for prototyping. A single HTTP GET call returns road distance (meters) and estimated driving time (seconds) between two coordinates.

```
GET /route/v1/driving/{station_lng},{station_lat};{fire_lng},{fire_lat}
→ { "routes": [{ "distance": 3200, "duration": 420 }] }
```

4.2 Call Pattern

The OSRM call is made through a **routing service** (backend/services/routing.py), not directly in the route handler:

- **Synchronous attempt** — the routing service calls OSRM with station coords → fire location coords at submission time.
- **Async fallback** — if the call fails (timeout, network error, OSRM down), enqueue a Celery task (tasks.routing) to retry. The report submission always succeeds regardless.
- **PostGIS fallback** — if both OSRM attempts fail, use existing ST_Distance straight-line distance × 1.5 sinuosity factor as a rough estimate. Duration is derived as straight_line_distance × 1.5 / 11.11 m/s (40 km/h average urban speed).

4.3 New Columns on citizen_reports

Column	Type	Nullable	Purpose
routing_distance_m	FLOAT	Yes	Road distance in meters from OSRM or fallback
routing_duration_s	FLOAT	Yes	Estimated driving time in seconds
routing_data_source	TEXT	Yes	"osrm" or "postgis_straight_line"
routing_execution_path	TEXT	Yes	"sync", "celery", or "fallback"
routing_updated_at	TIMESTAMPTZ	Yes	When routing was computed

routing_data_source tracks whether the values came from OSRM or PostGIS estimation. routing_execution_path tracks whether the sync call succeeded, a Celery retry succeeded, or both failed and we fell back. This split avoids the ambiguity of a single routing_provider column.

4.4 Frontend Display

The ETA range is derived client-side from routing_duration_s:

- Apply ±30% traffic buffer → e.g., 420s → "5–10 minutes"
- Below 180s: "Under 5 minutes"
- Above 1800s: "30+ minutes"

Station name, phone, distance, and ETA shown on the tracking page, dashboard, and validator panel.

4.5 Route Geometry (Deferred)

Explicitly not stored. The route geometry from OSRM is a stale snapshot by the time a future station-dispatch layer needs it — fresh routing based on current truck position and traffic will be computed

dynamically. Storing geometry adds 2-10 KB per route with no long-term value.

---

5. Anonymous vs Registered Model

Both paths are always available. Auth detection at the endpoint level determines which guardrails apply.

Property	Anonymous	Registered (CIVILIAN_REPORTER)
Identity	device_id (localStorage UUID)	Keycloak JWT
Auth on endpoints	None (public)	Depends(optional_auth) — returns user or None
Rate limit	3 reports/hour/IP	20 reports/hour
CAPTCHA	Required (Cloudflare Turnstile)	Skipped
Photos per report	Max 1, 5MB each	Max 5, 10MB each
Report visibility	Single tracking page	Full dashboard with history
Trust scoring	Single-report score only	Accumulated 0-100 score
Routing data	Station name + phone + distance + ETA	Same + dispatch history
Community page	Read-only announcements + station directory	Deferred to Phase 5

---

6. Contributor Trust Score

6.1 Formula (Hybrid Model)

```
trust_score = max(0, min(100, volume_credit + accuracy_bonus + photo_bonus - decay))
```

Component	Detail	Cap
Volume	+2 per report submitted	40 (20 reports)
Accuracy	+5 per report that reaches ACTIONED status	No cap
Photo bonus	+5 per report with photos, +10 if GPS consensus matches, +5 if photo near fire	20 per report
Decay	−2 per month of inactivity	Floor 0 (score never negative)

6.2 Badge Levels

Range	Badge
0–19	Novice
20–49	Regular
50–79	Trusted
80–100	Guardian

6.3 Photo Bonus Detail

Per Oracle recommendation — photos directly improve the report's trust score:

```
# Per-report photo bonus (aggregated across all photos on the report)
if photo_count > 0:
score += 5
if best_photo_gps_consensus == "both_match":
score += 10 # EXIF + browser GPS agree
if worst_photo_distance_to_fire < 500:
score += 5 # photo taken near the reported location
```

Maximum photo bonus per report: 20 points.

6.4 Storage

Trust score is **computed live** from report history at read time (simple aggregation query). No dedicated score table needed for the prototype — the computation is 3-5 SQL aggregations and a 10-line Python function. Cache with Redis if query latency becomes an issue.

---

7. Photo Upload + Metadata Analysis

7.1 Pipeline Flow

```
Civilian takes/selects photo
■
▼
navigator.geolocation.getCurrentPosition() ← GPS captured at the moment
of taking (camera) or selecting
(gallery). For gallery selections,
this records where the user is
now, which may differ from capture
location — the validator sees this
distinction.
■
▼
POST /api/civilian/photos/upload
multipart/form-data: file + browser_gps fields
■
▼
Server pipeline:
1. Sanitize filename + check extension + magic byte sniff
2. Read raw bytes
3. Extract EXIF from raw bytes ← BEFORE strip
4. Strip EXIF via strip_image_exif()
5. Compute MD5 hash of stripped bytes
6. Encrypt stripped bytes with AES-256-GCM (reusing get_crypto_provider())
7. Write encrypted file to disk
8. Insert report_photos row with EXIF data + browser GPS + metadata
■
▼
Returns { photo_id: "uuid-..." }
■
▼
POST /api/civilian/reports { ..., photo_ids: ["uuid-1", "uuid-2"] }
→ Backend validates photo_ids belong to this uploader (device_id or user_id)
→ Sets report_photos.report_id, computes gps_consensus + photo_reported_distance_m
```

7.2 report_photos Table

```
CREATE TABLE IF NOT EXISTS wims.report_photos (
photo_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
report_id INT REFERENCES wims.citizen_reports(report_id),
```

```
-- Uploader identity (one of these is set; the other is NULL)
uploader_user_id UUID REFERENCES wims.users(user_id), -- for registered
uploader_device_id TEXT, -- for anonymous

-- File metadata
original_filename TEXT NOT NULL,
storage_path TEXT NOT NULL,
file_size_bytes INTEGER NOT NULL,
mime_type TEXT NOT NULL,
image_width INTEGER,
image_height INTEGER,
md5_hash TEXT NOT NULL,

-- Encryption metadata (AES-256-GCM, reuses incident_attachments pattern)
encryption_iv TEXT,
encryption_key_version TEXT,

-- EXIF GPS (extracted before strip)
exif_gps_lat NUMERIC(10,7),
exif_gps_lon NUMERIC(10,7),
exif_datetime_original TIMESTAMPTZ,
exif_data JSONB,

-- Browser GPS (captured at photo selection time)
browser_gps_lat NUMERIC(10,7),
browser_gps_lon NUMERIC(10,7),
browser_gps_accuracy NUMERIC,
browser_gps_captured_at TIMESTAMPTZ,

-- Computed at submission time
photo_reported_distance_m NUMERIC,
gps_consensus TEXT, -- both_match | exif_only | browser_only | both_disagree | unavailable

-- Tracking
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
attached_at TIMESTAMPTZ
);

CREATE INDEX idx_report_photos_report_id ON wims.report_photos(report_id);
CREATE INDEX idx_report_photos_orphans ON wims.report_photos(created_at) WHERE report_id IS
NULL;
CREATE INDEX idx_report_photos_uploader ON wims.report_photos(uploader_user_id) WHERE
uploader_user_id IS NOT NULL;
```

7.3 Validation Logic

Thresholds:

- EXIF GPS vs Browser GPS difference: Dynamic threshold = max(100, browser_gps_accuracy × 3) meters
- Photo GPS (best available) vs reported fire location: 500m threshold
- If both GPS sources available and disagree → gps_consensus = "both_disagree"
- If neither available → gps_consensus = "unavailable"

Missing EXIF fallback: If EXIF GPS is unavailable (browser stripped it, common on mobile), trust the browser GPS. Only flag if both are missing or both disagree.

Extraction ordering is critical: EXIF extraction must happen BEFORE strip_image_exif() — the strip is destructive. The existing strip_image_exif in incidents.py will be reused as a utility function.

Browser GPS trust note: For gallery photo selections, the browser GPS marks *where the user is when they selected the photo*, not necessarily where the photo was taken. The validator sees both EXIF GPS

(capture location, if present) and browser GPS (selection location) independently.

7.4 Storage & Encryption

Photos are stored **encrypted at rest** using the existing AES-256-GCM infrastructure (get_crypto_provider() from services/kms.py), matching the incident_attachments encryption pattern. This is required because photos may contain PII (faces, license plates) and WIMS architecture mandates PII encrypted at rest.

Encryption metadata (encryption_iv, encryption_key_version) is stored on each report_photos row. The AAD binds ciphertext to photo:{photo_id}.

7.5 Ownership Enforcement

The uploader_user_id and uploader_device_id columns ensure:

- Anonymous photo uploads are bound to the caller's device_id
- Registered photo uploads are bound to their user_id
- The report submission endpoint validates that every photo_id in the request belongs to the caller's device_id or user_id
- Photo caps (anonymous max 1, registered max 5) are enforced by counting existing photos with matching uploader identity AND null report_id

7.6 Orphan Cleanup

A Celery beat task (tasks.cleanup_orphan_photos) deletes photos where report_id IS NULL AND created_at < now() - interval '24 hours'. Both the DB row and the encrypted physical file are removed.

---

8. Endpoint Architecture

8.1 Auth Detection

The existing get_current_user dependency raises 401 when no auth cookie is present — it cannot be used for endpoints that serve both anonymous and authenticated callers.

A new **optional_auth** dependency is needed for civilian routes. Unlike get_current_user, it catches the missing-auth condition silently and returns None:

```
async def optional_auth(request: Request, db: Session = Depends(get_db)):
"""Like get_current_wims_user but returns None instead of 401 on missing auth."""
try:
return await get_current_wims_user(request, db)
except HTTPException as exc:
if exc.status_code == 401:
return None
raise
```

This allows a single endpoint to branch on whether the caller is a registered contributor.

8.2 Same Endpoints, Auth Detection Pattern

The report submission endpoint detects authentication automatically:

```
# Pseudocode — auth detection in civilian.py
async def submit_civilian_report(
body: CivilianReportCreate,
request: Request,
db: Session,
```

```
user: Annotated[dict | None, Depends(optional_auth)] = None,
):
is_registered = user is not None and user.get("role") == "CIVILIAN_REPORTER"

if is_registered:
# Fast-track: higher rate limit, no CAPTCHA, link to account
rate_limit_cap = REGISTERED_REPORT_HOURLY_CAP # 20
contributor_user_id = user["user_id"]
else:
# Anonymous: strict limits, CAPTCHA required
await verify_turnstile(body.turnstile_token, request.client.host)
rate_limit_cap = CIVILIAN_REPORT_HOURLY_CAP # 3
body.device_id required
contributor_user_id = None
```

8.3 Complete Endpoint Set

Public (no auth):

Method	Endpoint	Purpose
POST	/api/civilian/photos/upload	Pre-upload photo (CAPTCHA required for anon)
POST	/api/civilian/reports	Submit report (CAPTCHA required for anon)
GET	/api/civilian/reports/{id}	Tracking page
PATCH	/api/civilian/reports/{id}/append	Append to report (CAPTCHA for anon)
GET	/api/civilian/announcements	BFP announcements
GET	/api/ref/fire-stations	Station directory (exists already)

Authenticated (requires CIVILIAN_REPORTER JWT):

Method	Endpoint	Purpose
GET	/api/civilian/contributor/me	Profile + trust score + badge
GET	/api/civilian/contributor/reports	Paginated report history
GET	/api/civilian/contributor/stats	Vanity metrics + chart data
GET	/api/civilian/contributor/leaderboard	Top contributors

8.4 Photo ID References

Photos use UUIDs natively. PostgreSQL accepts UUID[] and ANY(:ids) for bulk updates:

```
UPDATE wims.report_photos
SET report_id = :rid, attached_at = now()
WHERE photo_id = ANY(:photo_ids)
AND (uploader_user_id = :uid OR uploader_device_id = :did)
```

The WHERE clause enforces ownership — a caller can only attach photos they uploaded.

---

9. CAPTCHA Integration (Cloudflare Turnstile)

9.1 Scope

Turnstile is required on anonymous submissions for:

- POST /api/civilian/reports
- POST /api/civilian/photos/upload
- PATCH /api/civilian/reports/{id}/append

Skipped entirely for registered contributors.

9.2 Integration

Frontend: Drop the Turnstile widget into the report form:

```
<div class="cf-turnstile" data-sitekey="{TURNSTILE_SITE_KEY}"></div>
```

Backend: Verify the token server-side before processing:

```
POST https://challenges.cloudflare.com/turnstile/v0/siteverify
Body: { "secret": "{TURNSTILE_SECRET_KEY}", "response": "{token}" }
→ { "success": true/false }
```

Env vars: TURNSTILE_SITE_KEY (frontend) + TURNSTILE_SECRET_KEY (backend).

Development: Test keys that always pass: 1x00000000000000000000AA / 1x0000000000000000000000000000000AA.

---

10. Validator Review Enrichment

10.1 What the Validator Sees

In the triage/cluster inspection modal, the validator sees per-report:

- Routing data: Quezon City Fire Station 1 — 3.2 km road distance, est. 8-18 min ETA
- Photo metadata per photo:
- GPS consensus status: "both_match", "exif_only", "browser_only", "both_disagree", "unavailable"
- Photo distance from reported fire location
- Device make/model (from EXIF)
- Photo timestamp vs report timestamp comparison
- Trust score for registered contributors (displayed as badge level)

10.2 Validator Feedback

Validators can include routing and photo metadata context in their terminal action explanations (already supported — status_explanation field on terminal actions).

---

11. Pydantic Schema Changes

Existing schemas need new fields. Key changes:

CivilianReportResponse

New Field	Type	Purpose
photo_count	int	Number of attached photos
submitter_type	str	"anonymous" or "registered"

PhotoUploadResponse

Field	Type	Purpose
photo_id	str	UUID of the uploaded photo
file_size_bytes	int	File size
mime_type	str	Image MIME type
exif_gps_status	str	"present" or "unavailable"
browser_gps_status	str	"present" or "unavailable"

ContributorProfileResponse

Field	Type	Purpose
username	str	Keycloak username
trust_score	int	Current 0-100 score
badge	str	"novice", "regular", "trusted", "guardian"
total_reports	int	Lifetime report count
verified_reports	int	Reports with ACTIONED status
registered_since	datetime	Account creation date

---

12. Database Changes Summary

12.1 citizen_reports — 6 new columns (5 routing + 1 contributor FK)

```
ALTER TABLE wims.citizen_reports ADD COLUMN routing_distance_m FLOAT;
ALTER TABLE wims.citizen_reports ADD COLUMN routing_duration_s FLOAT;
ALTER TABLE wims.citizen_reports ADD COLUMN routing_data_source TEXT;
ALTER TABLE wims.citizen_reports ADD COLUMN routing_execution_path TEXT;
ALTER TABLE wims.citizen_reports ADD COLUMN routing_updated_at TIMESTAMPTZ;
ALTER TABLE wims.citizen_reports ADD COLUMN contributor_user_id UUID REFERENCES
wims.users(user_id);
```

contributor_user_id is NULL for anonymous reports, set to the user's UUID for registered contributors.

12.2 report_photos — new table

As specified in §7.2.

RLS: New report_photos table needs RLS policies bound to wims.current_user_id GUC, matching the pattern used by citizen_reports. The unauthenticated photo upload endpoint requires a SECURITY DEFINER helper or BYPASSRLS since there's no current_user_id at that point — same pattern as the existing public DMZ endpoints.

Audit: Photo uploads and photo-report attachments produce PHOTO_UPLOAD and PHOTO_ATTACH_ action entries in wims.audit_log.

12.3 users — no change

The CIVILIAN_REPORTER role already exists. No schema changes needed for the contributor model.

13. Service Layer

Business logic is placed in services, not routes, following WIMS architecture constraints.

Service File	Responsibility
services/routing.py	OSRM API calls, PostGIS fallback, write routing columns
services/report_photos.py	EXIF extraction, encryption, upload + attach logic, orphan cleanup
services/contributor.py	Trust score computation, leaderboard queries, dashboard aggregation

Celery tasks:

Task	Purpose
tasks.routing.retry_routing(report_id)	Async OSRM retry when sync call fails
tasks.routing.cleanup_orphan_photos()	Delete unattached photos >24h old

14. Env Vars

Variable	Used By	Purpose
TURNSTILE_SITE_KEY	Frontend	Turnstile widget site key
TURNSTILE_SECRET_KEY	Backend	Turnstile server-side verification
CIVILIAN_PHOTO_STORAGE_DIR	Backend	Photo storage path (default: /app/storage/civilian-photos)
REGISTERED_REPORT_HOURLY_CAP	Backend	Rate limit for registered contributors (default: 20)

15. Deferred Items

Item	Reason
Route geometry storage	Not useful — stale snapshot, future dispatch needs fresh routing
Forum/discussion on community page	Scope reduction for prototype
Badges/achievements	Scope reduction for prototype
Community page (leaderboard, safety content, events)	Deferred to Phase 5; registered users see announcements + station directory only until then
Nginx bot-blocker (bad bots)	Separate issue — nginx-ultimate-bad-bot-blocker
ETA with time-of-day factor	Over-engineered for prototype — ±30% traffic buffer is sufficient

16. Implementation Order

- **Phase 1 — Schema + OSRM routing** (5 new columns, routing service, OSRM call, Celery fallback)
- **Phase 2 — Photo upload pipeline** (new table, EXIF extraction, encryption, upload + attach endpoints, orphan cleanup)
- **Phase 3 — CAPTCHA** (Turnstile frontend + backend verification, optional_auth dependency)
- **Phase 4 — Registered contributor endpoints** (dashboard, profile, stats, leaderboard)
- **Phase 5 — Community page** (leaderboard, safety content, events)
- **Separate issue — Nginx bot-blocker**